



National University of Modern Languages

Assignment No 2

Name:	Tayyaba Ejaz (2751)
Course:	Computer Graphics
Class:	BSCS VII-A
Due Date:	7th -April, 2025
Submitted To:	Mam Sabina Irum

Table of Contents

Task: Apply various concepts & related to 2d and 3d Computer Graphics.....	3
1.1 Code	4
1.2 Output.....	10
1.3 Description	10

Task: Apply various concepts & related to 2d and 3d Computer Graphics.

Scenario:

You have been hired as a graphics developer for a new 2D park simulation game. Your task is to create a mini-park scene with multiple objects and apply transformations to bring it to life.

Assignment Tasks:

Object Creation:

Design and render at least three different objects (e.g., a tree, a bench, a pond, or a swing).

Use OpenGL/GLUT to create these objects using basic shapes (triangles, rectangles, circles, etc.).

Transformations:

Implement scaling to resize at least one object (e.g., growing or shrinking a tree).

Apply translation to move at least two objects to different positions (e.g., repositioning a bench).

Use rotation to animate an object (e.g., rotating a swing slightly to mimic movement).

Reflection (Bonus Task):

Implement reflection to show a water reflection for the pond or a mirrored object in the scene.

Multiple Viewports (Optional Extra Credit):

Divide the window into two viewports:

One showing the original park scene.

The other displaying a modified version with different transformations applied.

Submission Requirements:

- Submit a **C++ OpenGL program** implementing the above requirements.
- Include **comments** explaining the transformations applied.
- Submit at least **two screenshots**:

One before transformations.

One after transformations.

1.1 Code

```
#include <iostream>
#include <Windows.h>
#include <gl/Gl.h>
#include <gl/glut.h>
void MyInit() {
    glClearColor(1.0, 1.0, 1.0, 1.0);
    glColor3f(1.0, 0.0, 0.0);
    glPointSize(4.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 1000.0, 0.0, 600.0);
}
void pond() {
    glBegin(GL_POLYGON);
    glColor3f(0.0, 0.0, 2.0);
    glVertex2i(0, 100);
    glVertex2i(500, 200);
    glVertex2i(500, 0);
    glVertex2i(0, 0);
    glEnd();
}
void bench() {
    //back
    glBegin(GL_POLYGON);
    glColor3f(0.5, 0.2, 0.0);
    glVertex2i(120, 340);
    glVertex2i(340, 340);
    glVertex2i(340, 270);
    glVertex2i(120, 270);
    glEnd();
    //seat
    glBegin(GL_POLYGON);
    glColor3f(0.6, 0.3, 0.1);
    glVertex2i(120, 270);
    glVertex2i(340, 270);
    glVertex2i(300, 230);
    glVertex2i(80, 230);
    glEnd();
    //right leg
    glBegin(GL_POLYGON);
```

```

    glColor3f(0.5, 0.2, 0.0);
    glVertex2i(250, 230);
    glVertex2i(270, 230);
    glVertex2i(270, 180);
    glVertex2i(250, 180);
    glEnd();
    //left leg
    glBegin(GL_POLYGON);
    glColor3f(0.5, 0.2, 0.0);
    glVertex2i(130, 230);
    glVertex2i(150, 230);
    glVertex2i(150, 180);
    glVertex2i(130, 180);
    glEnd();

}

void tree() {
    //trunk
    glBegin(GL_POLYGON);
    glColor3f(0.6, 0.3, 0.1);
    glVertex2i(60, 260);
    glVertex2i(90, 260);
    glVertex2i(90, 380);
    glVertex2i(60, 380);
    glEnd();

    //triangle
    glBegin(GL_TRIANGLES);
    glColor3f(0.0, 0.6, 0.0);
    glVertex2i(20, 380);
    glVertex2i(130, 380);
    glVertex2i(75, 460);
    glEnd();

    glBegin(GL_TRIANGLES);
    glColor3f(0.0, 0.6, 0.0);
    glVertex2i(35, 420);
    glVertex2i(115, 420);
    glVertex2i(75, 500);
    glEnd();

```

```
glBegin(GL_TRIANGLES);  
glColor3f(0.0, 0.6, 0.0);  
glVertex2i(50, 470);  
glVertex2i(100, 470);  
glVertex2i(75, 530);  
glEnd();  
}
```

```
void swing() {  
    //stand  
    glBegin(GL_POLYGON);  
    glColor3f(0.0, 0.0, 0.0);  
    glVertex2i(380, 430);  
    glVertex2i(385, 430);  
    glVertex2i(360, 270);  
    glVertex2i(355, 270);  
    glEnd();
```

```
    glBegin(GL_POLYGON);  
    glColor3f(0.0, 0.0, 0.0);  
    glVertex2i(380, 430);  
    glVertex2i(385, 430);  
    glVertex2i(385, 270);  
    glVertex2i(380, 270);  
    glEnd();
```

```
    glBegin(GL_POLYGON);  
    glColor3f(0.0, 0.0, 0.0);  
    glVertex2i(470, 430);  
    glVertex2i(475, 430);  
    glVertex2i(475, 270);  
    glVertex2i(470, 270);  
    glEnd();
```

```
    glBegin(GL_POLYGON);  
    glColor3f(0.0, 0.0, 0.0);  
    glVertex2i(470, 430);  
    glVertex2i(475, 430);  
    glVertex2i(495, 270);
```

```

glVertex2i(500, 270);
glEnd();

glBegin(GL_POLYGON);
glColor3f(0.0, 0.0, 0.0);
glVertex2i(380, 430);
glVertex2i(475, 430);
glVertex2i(475, 437);
glVertex2i(380, 437);
glEnd();

//swing
glBegin(GL_POLYGON);
glColor3f(1.0, 1.0, 1.0);
glVertex2i(395, 430);
glVertex2i(400, 430);
glVertex2i(400, 300);
glVertex2i(395, 300);
glEnd();

glBegin(GL_POLYGON);
glColor3f(1.0, 1.0, 1.0);
glVertex2i(460, 430);
glVertex2i(465, 430);
glVertex2i(465, 300);
glVertex2i(460, 300);
glEnd();

glBegin(GL_POLYGON);
glColor3f(1.0, 1.0, 1.0);
glVertex2i(395, 300);
glVertex2i(465, 300);
glVertex2i(465, 305);
glVertex2i(395, 305);
glEnd();
}
void leftside() {
    glBegin(GL_POLYGON);
    glColor3f(0.0, 0.7, 1.0);
    glVertex2i(0, 600);
    glVertex2i(500, 600);

```

```

glVertex2i(500, 0);
glVertex2i(0, 0);
glEnd();
pond();
bench();
tree();
swing();
}
void transformedside() {
    glBegin(GL_POLYGON);
    glColor3f(0.6, 0.8, 1.0);
    glVertex2i(500, 600);
    glVertex2i(1000, 600);
    glVertex2i(1000, 0);
    glVertex2i(500, 0);
    glEnd();

    //pond
    glPushMatrix();
    glTranslatef(500.0, 0.0, 0.0);
    pond();
    glPopMatrix();

    //swing
    glPushMatrix();
    glTranslatef(150.0, 40.0, 0.0);
    glScalef(1.0, 1.1, 1.0);
    swing();
    glPopMatrix();

    //bench
    glPushMatrix();
    glTranslatef(530.0, 0.0, 0.0);
    glScalef(1.1, 1.1, 1.0);
    bench();
    glPopMatrix();

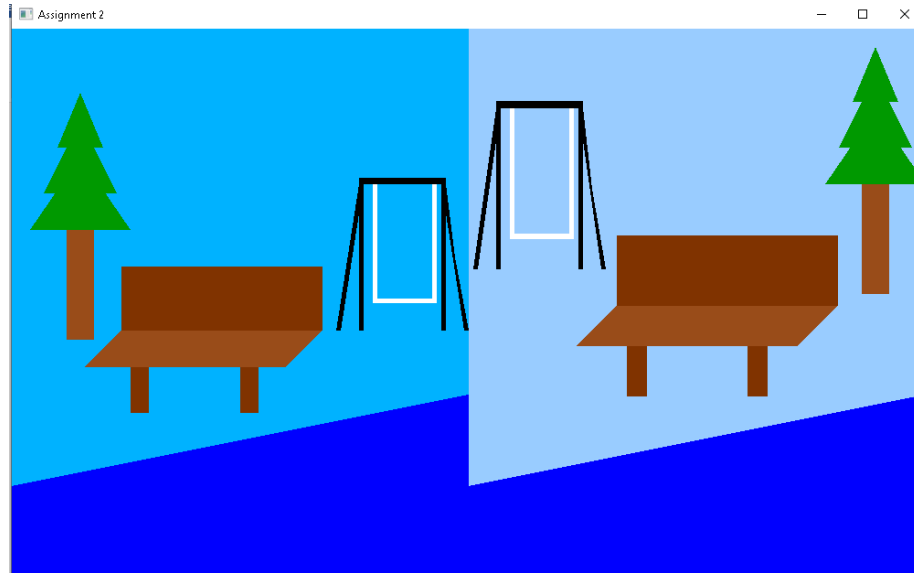
    //tree
    glPushMatrix();
    glTranslatef(870.0, 50.0, 0.0);
    tree();

```



```
    glPopMatrix();  
}  
  
void mydisplay() {  
    glClear(GL_COLOR_BUFFER_BIT);  
    leftside();  
    transformedside();  
    glFlush();  
}  
int main(int argc, char* argv[]) {  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowSize(1000, 600);  
    glutInitWindowPosition(150, 100);  
    glutCreateWindow("Assignment 2");  
    glutDisplayFunc(mydisplay);  
    MyInit();  
    glutMainLoop();  
}
```

1.2 Output



1.3 Description

This OpenGL program creates a simple 2D park scene divided into two parts. The left side displays the original objects including a pond, bench, tree, and swing, while the right side shows transformed versions using translation and scaling. Basic OpenGL functions like `glTranslatef`, `glScalef`, and matrix stack operations are used to manipulate the objects, demonstrating how transformations affect the appearance and position of elements in a 2D graphical environment.
